P4P / COMPSYS 770 A/B

Research Compendium

Project Overview & Evidence Index

Architectural Exploration and Voltage–Frequency Characterisation of a Capsule Neural Network on the ZCU102 FPGA

2026 • Department of Electrical, Computer, and Software Engineering

The University of Auckland, New Zealand

AUTHORS & RESEARCH TEAM	GUIDANCE
Tonny Chen | Student Beren Eskildsen | Project partner Josiah Sterling | Research partner	Maryam Hemmati | Supervisor Morteza Biglari-Abhari | Co-supervisor

Document role

This document explains the project, identifies evidence and artefacts, and directs future researchers to the accompanying setup, source-code, experimental-procedure and raw-data materials.

How to use this compendium

Start here if you are an examiner reviewing the evidence, a student rebuilding the accelerator, or a researcher extending the voltage–frequency study. The archive is organised by task: navigation and context live in this document; detailed procedures and primary evidence live in separate directories.

Reader pathways

Your goal	Start here	Then consult
Understand the study	01_Main_Compendium/Research_Compendium.pdf	Final reports and 03_System_Architecture/
Build and run CapsNet	02_Software_Setup_Guide/Software_Setup_Guide.pdf	04_Source_Code/ and the board deployment artefacts
Repeat a measurement	05_Experimental_Procedures/	06_Experimental_Data/ and 07_Verification_and_Results/
Audit a plotted result	Evidence traceability table in this document	Raw data, analysis scripts, generated figure
Continue the research	10_Future_Development/	Design source, experiment logs, known limitations

Document map

Document / material	Purpose
Research Compendium (this document)	A navigable overview, reproduction route, study results and evidence crosswalk.
Software Setup Guide	Fresh Ubuntu installation through first successful board inference.
Experimental Procedures	Exact stimulus, voltage sweep, measurements, tests, acceptance criteria.
Source Code and Build Scripts	Original model, V1/V2/V3, DPU/HLS integrations, host applications.
Experimental Data and Verification	Raw logs, board records, build reports, analysis scripts and figure outputs.

1 Project overview

1.1 Research question and contributions

This study examines how architectural specialisation of functionally equivalent DigitCaps accelerators changes nominal performance, FPGA resource utilisation, and the experimentally observed voltage–frequency reliability and energy characteristics on the ZCU102. The underlying routing-by-agreement workload is retained while memory organisation, parallelism, arithmetic precision and reduction structure are varied.

- Three DigitCaps HLS accelerators (V1–V3) were developed and compared under consistent implementation and validation conditions.
- Two ZCU102 boards were used for real-silicon voltage–frequency characterisation of the standalone accelerator builds.
- The joint architecture and operating-point comparison identifies the lowest measured energy configuration within the evaluated design space.

1.2 Quantitative research snapshot

Metric	Finding	Measurement context
V1 → V3 kernel latency	28.490 → 6.333 ms	DigitCaps, 150 MHz; integrated CapsNet configuration
Software baseline kernel latency	20.872 ms	Cortex-A53 @ 1.2 GHz
V3 full-network latency	10.749 ms at 150 MHz; 7.599 ms at 300 MHz	Integrated CapsNet configuration
Reference / V3 MNIST accuracy	99.66% / 98.88%	10,000-image test set
Lowest observed energy	9.76 mJ/inference	V3 standalone @ 300 MHz, 0.61 V; seven-rail boundary
Minimum tested supply	0.55 V	Experimental floor, not necessarily physical Vmin

1.3 Scientific scope and interpretation

The principal contribution is a controlled comparison across three differently specialised implementations of the same DigitCaps operator. The work does not claim to introduce a new dynamic-routing algorithm, nor a novel automatic runtime DVFS controller. Energy values measure a defined seven-rail board-level boundary rather than isolated DigitCaps core energy. Voltage limits are observed under the stated workload and test floor, not certified across all boards or environmental conditions .

2 System design and deployment

2.1 Hardware–software partitioning

The deployed CapsNet is split into six execution stages. Conv1 and Primary_conv2d run on the DPUCZDX8G via Vitis AI Runtime (VART); Primary_reshape is a data-layout operation; Primary_squash, DigitCaps and CapsNet_length are custom HLS accelerators invoked through XRT/ZOCL. DigitCaps transforms 1,152 eight-dimensional primary capsules into ten 16-dimensional output capsules using three routing iterations.

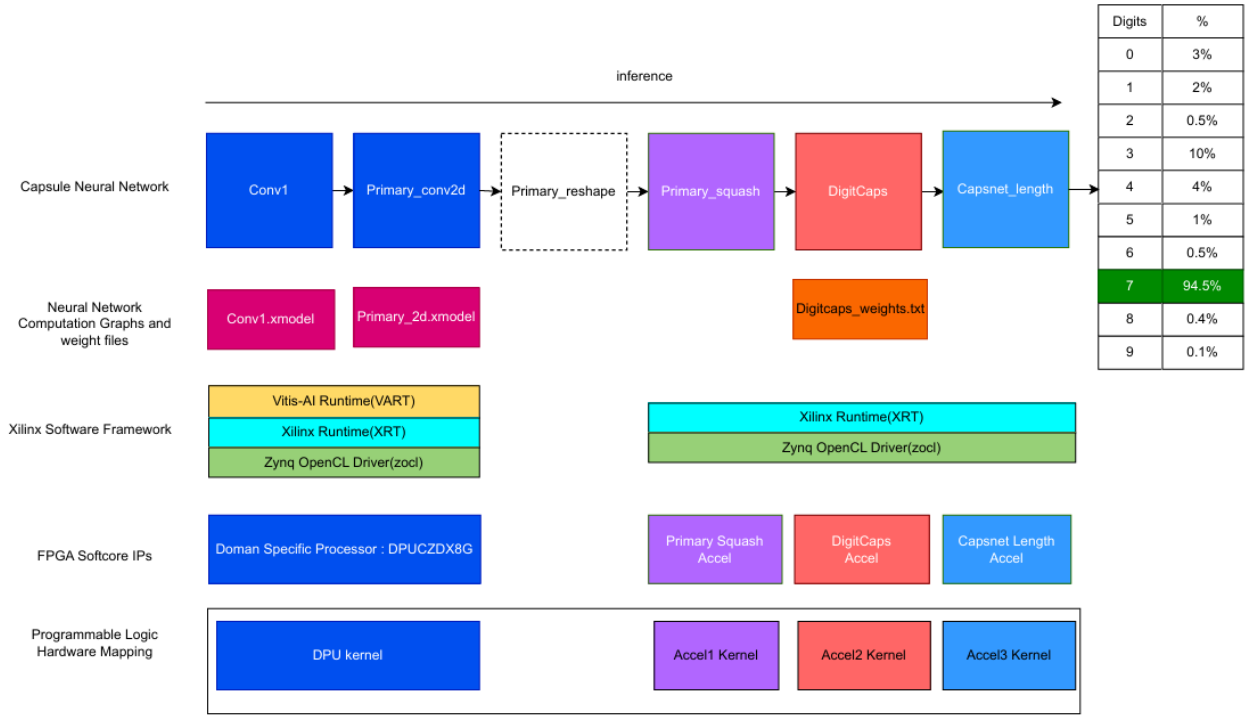


Figure 1. Runtime CapsNet partitioning (reproduced from the supplied Final Report, Fig. 1, p. 7).

2.2 Build and board deployment

The project uses parallel model and accelerator compilation paths. The convolutional portion is extracted from CapsNet.h5, quantised with Vitis AI 2.5 and compiled into XMODEL. The capsule-specific C++ implementations are synthesised with Vitis HLS 2022.1 into XO kernels; the DPU is packaged separately; v++ 2022.1 links the hardware kernels into an XCLBIN. The host application is cross-compiled using the PetaLinux 2022.1 SDK and deployed with the Linux runtime on an SD card.

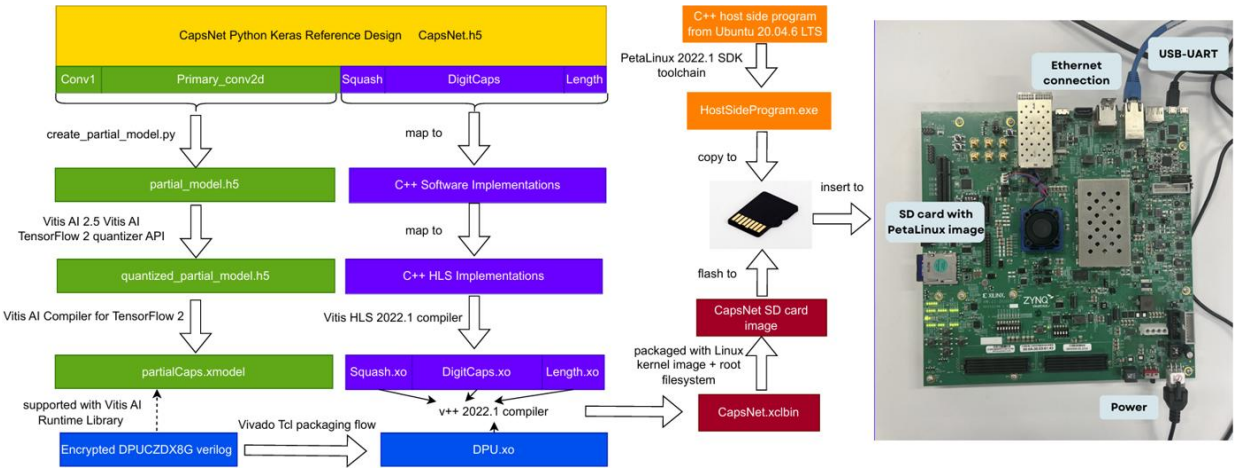


Figure 2. Model-to-board compilation and connection flow (supplied Final Report, Fig. 2, p. 8; board photograph credited there to the 2025 CES82 project).

2.3 Two build contexts must remain separate

Integrated CapsNet build	**Standalone DigitCaps build**
Contains DPU, Primary_squash, DigitCaps and CapsNet_length acceleration path.	Contains a standalone DigitCaps implementation for isolated measurement.
Use: end-to-end inference, MNIST classification accuracy, host overhead and integrated kernel latency.	Use: architecture resource utilisation, voltage–frequency reliability, energy and power characterisation.
Do not use these results as standalone DigitCaps energy estimates.	Do not claim whole-network inference latency from these measurements.

3 Design archive: V1, V2 and V3

The three implementations preserve the routing-by-agreement computation while changing hardware mapping. The design-history files should explain each major choice alongside the corresponding synthesised source, synthesis settings, logs and representative tests .

Design	Memory and numerical representation	Architectural intent
V1 — baseline	Wide fixed point; weights accessed from PS-side DDR4.	Conservative software-like pipelined baseline; relatively limited parallelism.
V2 — memory / parallelism	INT8 + scale; eight weights per 64-bit word cached in BRAM.	Localise weights, fully unroll prediction MAC, use parallel routing/reductions.
V3 — specialised datapath	7-bit fixed-point weights (ap_fixed<7,1>), banked BRAM; stage-specific precision.	Two 8-D prediction MAC trees, 16-way squash reduction; first-iteration softmax bypass and reciprocal reuse.

3.1 Physical implementation evidence

Implementation	LUT	FF	BRAM	DSP	Fmax
V1	21,702	26,200	260.5	52	159.6 MHz
V2	64,268	57,049	495	341	306.3 MHz
V3	33,223	26,978	417	147	306.1 MHz

Source: Final Report, Table 4, p. 15. Retain the original post-implementation Vivado/Vitis reports and exact physical-build settings, not only these summary values.

3.2 Functional accuracy and validation

The report gives 99.63%, 99.57%, and 98.88% MNIST test accuracy for V1, V2 and V3 respectively, compared with 99.66% for the reproduced Keras reference Nominal functional validation progressed from Python versus C++ comparison through HLS C/RTL co-simulation, hardware emulation, minimal-kernel board tests and full application checking .

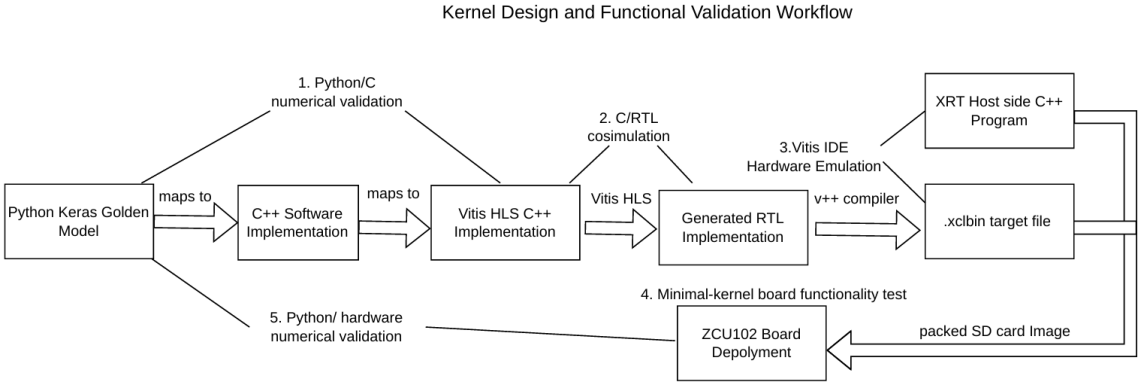


Figure 3. Functional validation chain (supplied Final Report, Fig. 6, p. 13).

4 Experiment design and measurements

4.1 Experimental conditions

Item	Recorded method / scope
Board population	Two ZCU102 boards; identifying serial numbers remain to be inserted in the archive.
Configurations	V1: 50, 100, 150 MHz. V2/V3: 50, 100, 150, 200, 250, 300 MHz.
Voltage sweep	VCCINT 0.85 V nominal down to 0.55 V floor in 0.01 V steps; other rails unchanged.
Input stimulus	First 100 MNIST images; corresponding PrimaryCaps tensors captured from the integrated system and reused for standalone kernels.
Warm-up	50 images before the main sweep.
Reference output	Each implementation's own 0.85 V hardware output; 160 DigitCaps values per image.
Reliability pass	All 100 × 160 outputs match the nominal-voltage hardware reference exactly.
Measurements	DigitCaps execution time and seven-rail board monitoring; temperature recorded alongside voltage sweeps.

Source: Final Report, §§5.1–5.2. The automated voltage scripts, timestamps, power sampling details, calibration/verification method and any settling intervals must be checked against the actual 2026 experiment scripts and added to the detailed Experimental Procedures guide.

4.2 Power measurement boundary

The same monitored-rail boundary was used for the standalone hardware and the Cortex-A53 software baseline: VCCINT, VCCBRAM, VCCAUX, VCCPSINTFP, VCCPSINTLP, DDR4_DIMM_VDDQ, and VCCO_PSDDR_504. The energy-per-inference result includes this measured system-level rail set, not just the VCCINT rail (Final Report, §5.2).

4.3 Measurement provenance

Future users must be able to locate the voltage setter and readback, the rail-sampling utility, raw timestamps and measurements, software reference output, board temperature records, input tensor capture and per-configuration binary SHA256 hashes. A record for each experiment should identify date, operator, board, kernel variant, target clock, git revision, XCLBIN, command and outcome.

5 Representative results

5.1 Voltage–frequency reliability

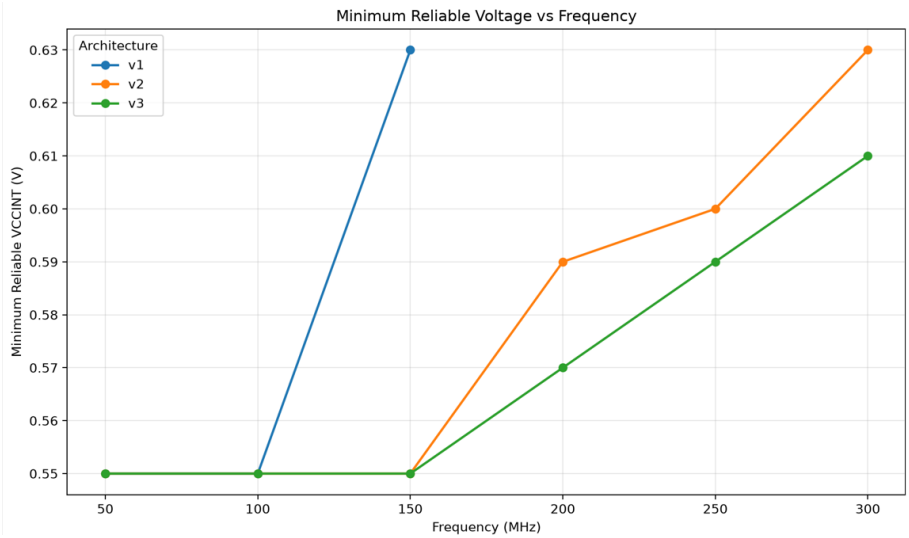


Figure 4. Observed reliable voltage floor by frequency (supplied Final Report, Fig. 8, p. 20). Results at 0.55 V are floor-bounded.

At 150 MHz, V2 and V3 remained reliable at the 0.55 V experimental floor while V1 required 0.63 V. The minimum observed reliable VCCINT for V3 at 300 MHz was 0.61 V (Final Report, §6.2).

5.2 Energy at reliable operating points

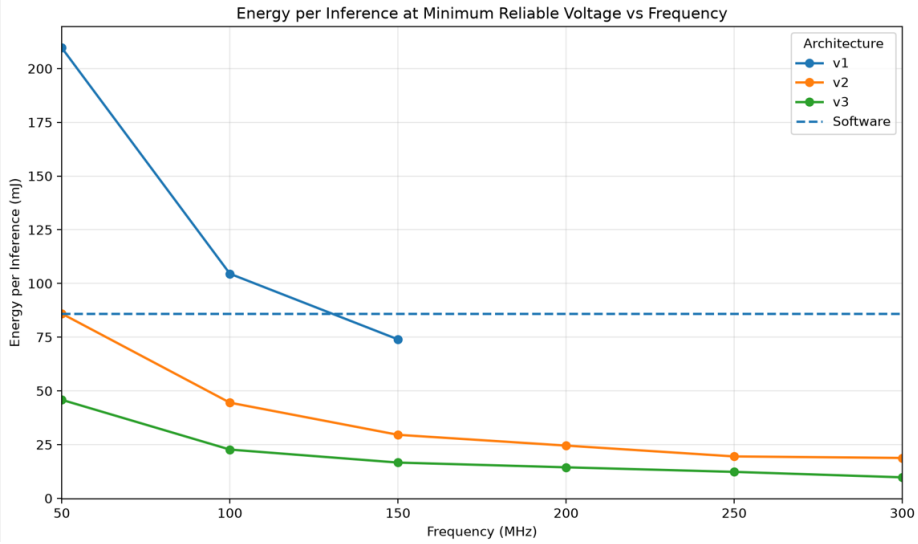


Figure 5. Seven-rail energy per inference at each tested reliable-voltage point (supplied Final Report, Fig. 9, p. 21).

At 150 MHz, the recorded energies were V1 73.96 mJ at 0.63 V, V2 29.55 mJ at 0.55 V and V3 16.62 mJ at 0.55 V. The lowest measured point in the evaluated design space was V3 at 300 MHz and 0.61 V (9.76 mJ/inference), compared with the 85.95 mJ/inference Cortex-A53 baseline (Final Report, §6.3).

6 Reproduction roadmap

The companion Software Setup Guide should provide executable commands and expected outputs. This overview specifies the order and acceptance gates; it deliberately does not invent unverified build commands. Preserve the exact project scripts as the authoritative implementation instructions.

Gate	Work to complete	Evidence of success
G0 — inventory	Confirm board, SD card, UART, Ethernet, host laptop and exact tool versions.	Board/equipment register; installed environment manifest.
G1 — workstation	Prepare Ubuntu 20.04.6 LTS, Vivado/Vitis/Vitis HLS 2022.1, Vitis AI 2.5 and PetaLinux SDK 2022.1.	Version commands, package manifests, toolchain output.
G2 — model	Extract convolutional sub-model; quantise and compile XMODEL against the DPU architecture.	Model and XMODEL artefacts; compilation logs.
G3 — accelerators	Build V1/V2/V3 HLS kernels; run reference and simulation tests.	XO files, C simulation / co-simulation logs.
G4 — integration	Link DPU and three custom kernels with platform and exact directives.	Timing-closed XCLBIN and implementation reports.
G5 — deployment	Cross-compile host application; prepare SD image; connect ZCU102 and boot.	Boot log, network/UART diagnostics, app binaries.
G6 — first inference	Run nominal inference; verify result and numerical output.	Captured terminal log, test stimulus and expected output.
G7 — experiments	Execute and record latency, accuracy, V–f and seven-rail energy experiments.	Raw results, metadata, criteria and reproducible plots.

6.1 Baseline toolchain

Host / target component	Reference identified in the report
Development workstation	Ubuntu 20.04.6 LTS
Vivado / Vitis / Vitis HLS	2022.1
Vitis AI and TensorFlow 2 flow	Vitis AI 2.5; compile with the TensorFlow 2 compiler
Host cross-compiler	PetaLinux 2022.1 SDK
Target	ZCU102 (XCZU9EG), DPUCZDX8G, XRT/VART runtime
Platform and images	Record the exact ZCU102 .xpfm, common-image/rootfs and XRT build in the setup guide

The report identifies the major tool versions, but not every package checksum, environment variable and command-line dependency. These must be captured in the setup guide/environment manifest; do not infer them from tool version alone.

7 Archive directory and evidence index

The following is the agreed compendium organisation. Relative paths are proposed archive targets: they become verified evidence links only after actual files are copied into the final ZIP and the root README is checked. Maintain the distinction between “planned”, “included” and “independently reproduced”.

Directory	Expected content	Primary user
01_Main_Compendium	This overview; final joint/individual reports; versioned report index.	Examiner / first-time reader
02_Software_Setup_Guide	Fresh-host setup, required packages, Xilinx/PetaLinux/Vitis AI, first inference, screenshots, troubleshooting.	New student / builder
03_System_Architecture	Runtime architecture, build and deployment, DigitCaps datapaths, high-resolution diagrams and hardware/software mapping.	Architect / developer
04_Source_Code	CapsNet model, HLS V1–V3 and testbenches, DPU/FPGA scripts, target applications, inputs/weights and build configuration.	Developer / re-builder
05_Experimental_Procedures	Latency/accuracy, V–f sweep, power/energy and repeatability protocols; stimulus and device records.	Experiment operator
06_Experimental_Data	Unmodified raw logs and samples; processed CSV; analysis and plotting scripts; figure crosswalk.	Data auditor / analyst
07_Verification_and_Results	Numerical reference comparisons, co-sim, timing/resource reports, board checks, calibration/verification and results.	Quality reviewer
08_Project_Records	Meeting/actions, progress logs, design decisions, issue records, project chronology, research notes.	Project reviewer
09_Individual_Submissions	Final Canvas-submitted files for both students with individual manifests.	Examiner / supervisor
10_Future_Development	Known limitations, unresolved questions, extensions and recommended starting points.	Future researcher

7.1 Recommended subfolder conventions

Location	Naming / treatment
04_Source_Code/DigitCaps_V1_V2_V3/	Separate V1/, V2/, V3/ with source, synthesis settings, tests and README; include a manifest tying each XO/XCLBIN to a commit.
06_Experimental_Data/Raw_Data/	Create board/variant/frequency/test-run groupings; never edit or overwrite raw files.
06_Experimental_Data/Processed_Data/	Only script-generated transformation outputs, with clear source filename references.
06_Experimental_Data/Plotting_Scripts/	Executable scripts and pinned Python dependencies; save regeneration commands.
09_Individual_Submissions/<student>/	Include exact Canvas versions and an explicit submission_manifest.md.

8 Evidence traceability and handover

8.1 Required experiment record

For each experiment, record a structured run ID and the following: operator, time and timezone, board serial, SD/image identifier, kernel architecture, requested and measured frequency, requested and read-back VCCINT, stimulus/weight versions, input count, command/arguments, reference policy, generated artefacts, pass/fail result, and any exceptions. For energy runs, include timestamps, all seven rail channels, filtering rules and temperature.

8.2 Provenance of prior and collaborative work

The supplied report states that this project builds upon the CSE033 2024 CapsNet-on-ZCU102 deployment, with a 2025 CES82 board image used in the report's build-flow figure. It credits Beren Eskildsen for automated undervolting scripts and Josiah Sterling for the undervolting infrastructure and support. The submission must retain acknowledgements, identify externally obtained model/DPU components, and distinguish inherited code from 2026 changes (Final Report, §§3.2–3.3 and Acknowledgements).

8.3 Separate individual submissions

The joint compendium must include both students' Canvas-submitted files, clearly assigned to each individual. Suggested paths: 09_Individual_Submissions/Tonny_Chen/ and 09_Individual_Submissions/Beren_Eskildsen/. Each student folder needs a short manifest listing the exact file, submitted version, submission date, and any shared material.

8.4 Minimum equipment register

Item	Identification to record	Status
ZCU102 board 1 / board 2	Serial number, FPGA board revision, firmware/boot-switch settings	TO SUPPLY
Host workstation	Model, OS/architecture, tool and installed version manifest	TO SUPPLY
Power monitoring	Rail-to-sensor mapping, source path and calibration/verification method	TO SUPPLY
Cables, SD card and network	Connection and storage details; SD image hash	TO SUPPLY

9 Known limitations and future development

9.1 Interpretation limits

- A voltage point passing 100 images provides an observed workload-dependent boundary, not an exhaustive fault-free guarantee.
- Runs at the 0.55 V test floor are censored by the experiment limit; the true failure threshold may be lower.
- The energy result is a seven-rail measurement and is not equivalent to accelerator-only dynamic power.
- The comparison covers two ZCU102 boards and this DigitCaps workload; wider device and operator generality has not been established.
- V3's improved latency and resource utilisation are accompanied by lower 10,000-image MNIST classification accuracy.

9.2 Research continuation opportunities

The report proposes extending the study to additional devices, finer and repeated voltage checks near measured boundaries, and a more systematic architecture-aware precision/parallelism/voltage design-space search. Extending the same testing infrastructure to other irregular neural-network operators could test whether architecture-sensitive reliability is a broader phenomenon rather than DigitCaps-specific (Final Report, §8.2).

10 Final archive audit before Canvas submission

Priority	Audit item	Completion
Critical	All ten top-level directories exist and README.md paths match exactly.	☐
Critical	Software Setup Guide leads from clean workstation to an independently checked first inference.	☐
Critical	All source, build scripts, DPU architecture configuration, model weights and required binaries are available or retrievable with exact versions.	☐
Critical	Experiment protocols and raw results substantiate every reported figure/table.	☐
Critical	Both individual Canvas submission sets are present and attributed.	☐
Important	Two ZCU102 serials, sensor mapping and measurement verification records are supplied.	☐
Important	Reproduction output is traceable to Git commit, configuration and SHA256 checksum.	☐
Important	Included design diagrams have legible source/credit captions.	☐
Important	ZIP size is below 2 GB or alternative supervisor submission is agreed.	☐

References and document provenance

Primary source for the numerical claims and technical workflow: Tonny Chen, "Architectural Exploration and Voltage–Frequency Characterisation of a Capsule Neural Network on the ZCU102 FPGA," uploaded 36-page final-year project report, 2026. Sections, table numbers, and figure numbers in this compendium refer to that supplied report. This overview has been compiled from the report and the agreed archive structure; it does not certify that the separate evidence files have already been added to the archive.

Public project history/repository: https://github.com/wolfuoa/P4P-CapsuleNetworkAccelerator . Preserve a pinned commit and state clearly which 2026 files are newer or different from the historical repository.